

# Tổng Hợp Kiến Thức

## 1. Chuẩn Bị và Cài Đặt

### Cài Đặt Thư Viện

Cài các gói cần thiết cho tải ảnh và bảo mật:

```
npm install multer sharp helmet express-rate-limit
```

Lưu ý:

- Đảm bảo `sharp` cài đặt thành công (có thể cần build tools trên Windows).
- Nếu `sharp` lỗi, tập trung vào `multer` để tải ảnh và bỏ qua resize.

## 2. Cấu Hình Multer để Tải Ảnh

**Middleware:** `uploadMiddleware.js`

Xử lý tải file với `multer`, bao gồm lưu trữ, lọc file và giới hạn kích thước.

```
// src/middlewares/uploadMiddleware.js
const multer = require('multer');
const path = require('path');

const multerStorage = multer.diskStorage({
  destination: (req, file, cb) => {
    const tempPath = path.join(__dirname, '..', 'public', 'temp');
    cb(null, tempPath);
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    const ext = path.extname(file.originalname);
    cb(null, `image-${uniqueSuffix}${ext}`);
  }
});

const multerFilter = (req, file, cb) => {
  if (file.mimetype.startsWith('image')) {
    cb(null, true);
  } else {
    cb(new Error('Không phải ảnh! Vui lòng chỉ tải lên file ảnh.'), false);
  }
};

const upload = multer({
  storage: multerStorage,
  fileFilter: multerFilter,
  limits: { fileSize: 5 * 1024 * 1024 } // Giới hạn 5MB
});

exports.uploadSingleImage = (fieldName) => upload.single(fieldName);
```

**Điểm chính:**

- `destination`: Lưu file tạm vào `src/public/temp`.
- `filename`: Tạo tên file duy nhất để tránh xung đột.
- `fileFilter`: Chỉ cho phép tải file ảnh.
- `limits`: Giới hạn kích thước file tối đa 5MB.

### 3. Xử Lý Ảnh với Sharp

**Middleware:** `imageProcessingMiddleware.js`

Xử lý ảnh tải lên bằng `sharp` để resize, chuyển định dạng và dọn dẹp file tạm.

```
// src/middlewares/imageProcessingMiddleware.js
const sharp = require('sharp');
const path = require('path');
const fs = require('fs');

exports.resizeImage = async (req, res, next) => {
  if (!req.file) return next();

  const tempFilePath = req.file.path;
  const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
  const finalFilename = `image-${uniqueSuffix}.jpeg`;
  req.file.processedFilename = finalFilename;

  const finalDirectory = path.join(__dirname, '..', 'public', 'uploads');
  const finalFilePath = path.join(finalDirectory, finalFilename);

  try {
    await sharp(tempFilePath)
      .resize(1000)
      .toFormat('jpeg')
      .jpeg({ quality: 80 })
      .toFile(finalFilePath);

    fs.unlink(tempFilePath, (err) => {
      if (err) console.error(`Lỗi xóa file tạm ${tempFilePath}:`, err);
      else console.log(`Đã xóa file tạm: ${tempFilePath}`);
    });

    next();
  } catch (err) {
    console.error('Lỗi xử lý ảnh với Sharp:', err);
    fs.unlink(tempFilePath, (unlinkErr) => {
      if (unlinkErr) console.error(`Lỗi xóa file tạm ${tempFilePath} sau lỗi xử lý:`, unlinkErr);
      else console.log(`Đã xóa file tạm sau lỗi: ${tempFilePath}`);
    });
    next(err);
  }
};
```

**Lưu ý:**

- Resize ảnh xuống 1000px chiều rộng, chuyển sang `.jpeg` với chất lượng 80%.
- Xóa file tạm sau khi xử lý để tránh rác.
- Lưu tên file đã xử lý vào `req.file.processedFilename` để dùng sau.

### 4. Cấu Hình Route và Controller

**Route:** `userRoutes.js`

Thêm route để xử lý tải ảnh đại diện.

```
// src/routes/userRoutes.js
const express = require('express');
const router = express.Router();
const authenticateToken = require('../middlewares/authMiddleware');
```

```
const uploadMiddleware = require('../middlewares/uploadMiddleware');
const imageProcessingMiddleware = require('../middlewares/imageProcessingMiddleware');
const userController = require('../controllers/userController');

router.patch(
  '/update-my-avatar',
  authenticateToken,
  uploadMiddleware.uploadSingleImage('avatar'),
  imageProcessingMiddleware.resizeImage,
  userController.updateMyAvatar
);

module.exports = router;
```

**Controller:** `userController.js`

Cập nhật avatar người dùng trong database.

```
// src/controllers/userController.js
const { User } = require('../models');
const path = require('path');

exports.updateMyAvatar = async (req, res, next) => {
  if (!req.file || !req.file.processedFilename) {
    return res.status(400).json({
      status: 'fail',
      message: 'Tải ảnh hoặc xử lý ảnh thất bại.'
    });
  }

  try {
    const avatarPath = `/uploads/${req.file.processedFilename}`;
    await User.update(
      { avatar: avatarPath },
      { where: { id: req.user.id } }
    );

    res.status(200).json({
      status: 'success',
      message: 'Cập nhật avatar thành công!',
      data: { avatarUrl: avatarPath }
    });
  } catch (err) {
    next(err);
  }
};
```

## Cấu Hình File Tĩnh trong `server.js`

Cho phép truy cập file tĩnh từ `public`.

```
// server.js
const express = require('express');
const app = express();
const path = require('path');

app.use(express.static(path.join(__dirname, 'public')));
app.use(express.json());
app.use('/api/users', require('../routes/userRoutes'));
```

# 5. Tăng Cường Bảo Mật

## Helmet

Thêm các header bảo mật HTTP để bảo vệ ứng dụng.

```
// server.js
const helmet = require('helmet');

app.use(helmet());
```

### Công dụng:

- Ngăn XSS, clickjacking, và các lỗ hổng web phổ biến.
- Ẩn thông tin server (VD: `X-Powered-By: Express` ).

## Express Rate Limit

Giới hạn số lượng request từ một IP.

```
// server.js
const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
  max: 100, // 100 requests mỗi 15 phút
  windowMs: 15 * 60 * 1000, // 15 phút
  message: 'Quá nhiều yêu cầu từ IP này, vui lòng thử lại sau 15 phút!',
  standardHeaders: true,
  legacyHeaders: false
});

app.use('/api', limiter);
```

### Lưu ý:

- Chỉ áp dụng cho endpoint `/api` để không ảnh hưởng file tĩnh.
- `max` : Số request tối đa.
- `windowMs` : Khung thời gian giới hạn.